

BẢN ĐẶC TẢ KỸ THUẬT TOÀN DIỆN V2

Quy hoạch và Chuyển đổi mã nguồn Lashing Hole từ VBA sang AutoCAD .NET API (C# / WPF)

1. TỔNG QUAN VÀ BẢN ĐỒ ÁNH XẠ THÀNH PHẦN (COMPONENT MAPPING)

Tài liệu này tích hợp toàn bộ các nghiệp vụ logic từ file gốc `CreateLashingHole.dvb`, bổ sung chi tiết cấu trúc cho **Khu vực đặc biệt (Phase 3)** và **Cơ chế quét cấu kiện liền kề (Adjacent Check)** vào mô hình kiến trúc **MVVM** của repository `SDS`.

Thành phần VBA gốc	Thành phần .NET / C# Mục tiêu	Vị trí Thư mục dự án SDS
Cấu trúc biến toàn cục	<code>LashingHoleModels.cs</code> (Strongly-typed POCO)	<code>Models/</code>
<code>frmLashingInput.frm/.frx</code>	<code>LashingHolePalette.xaml</code> (WPF UserControl)	<code>UI/</code>
<code>mod_LashingHoleInterference.bas</code>	<code>CollisionEngineService.cs</code> (Xử lý va chạm hình học RAM)	<code>Services/</code>
<code>mod_MainLashingHole_V3_5.bas</code>	<code>GridGenerationService.cs</code> & <code>BlockPackingService.cs</code>	<code>Services/</code>
<code>ModInputHelper_V3.bas</code>	<code>AutoCADGeometryHelper.cs</code> (Giao tiếp bản xử hạ tầng)	<code>Utilities/</code>
Lệnh gọi Macro	<code>LashingHoleCommands.cs</code> (Đăng ký lệnh CAD hệ thống)	<code>Commands/</code>

2. ĐỊNH NGHĨA MÔ HÌNH DỮ LIỆU (MODELS/ LASHINGHOLEMODELS.CS`)

Thiết lập cấu trúc dữ liệu hướng đối tượng nhằm thay thế hoàn toàn biến rải **Variant** lỏng lẻo bằng các kiểu dữ liệu an toàn (Strongly-typed):

```

namespace SDS.Models
{
    public enum LashingLocationMode
    {
        PortSide,    // Điểm neo P1 nằm tại góc Dưới - Trái
        StarBoard,   // Điểm neo P1 nằm tại góc Trên - Trái
        Center       // Tự động khởi rải điểm từ tâm hình học đối xứng của tấm biên
    }

    public enum LashingCollisionType
    {
        None,
        Vertical,
        Horizontal,
        Complex
    }

    public class LashingInputParams
    {
        public double HoleDiameter { get; set; } = 55.0;
        public double ClearanceRadius { get; set; } = 75.0;
        public double OffsetX { get; set; } = 150.0;
        public double OffsetY { get; set; } = 150.0;
        public double SpacingX { get; set; } = 500.0;
        public double SpacingY { get; set; } = 500.0;
        public string PanelName { get; set; } = "PNL_01";
        public LashingLocationMode LocationMode { get; set; } =
LashingLocationMode.PortSide;
        public bool IsAutomaticMode { get; set; } = false;
        public bool IsCheckAdjacent { get; set; } = false;
    }

    public class LashingCollisionResult
    {
        public bool CollisionOccurred { get; set; } = false;
        public LashingCollisionType CollisionType { get; set; } =
LashingCollisionType.None;
        public double DeltaX { get; set; } = 0;
        public double DeltaY { get; set; } = 0;
    }
}

```

3. LỚP XỬ LÝ THUẬT TOÁN NÉ VA CHẠM VÀ TIA ẢO (` SERVICES/`)

A. `CollisionEngineService.cs` - Bộ Máy Tính Toán Va Chạm & Cấu Kiến Liên Kề

Toàn bộ phép toán giao cắt được tính trực tiếp trong RAM thông qua đối tượng `CircularArc2d` và `Line3d` bản xứ, tối ưu hóa triệt để tài nguyên hệ thống:

```

using System;
using System.Collections.Generic;
using Autodesk.AutoCAD.DatabaseServices;
using Autodesk.AutoCAD.Geometry;

namespace SDS.Services
{
    public class CollisionEngineService
    {
        // Phân loại hướng va chạm dựa vào tỉ lệ Delta X và Delta Y hình học
        public LashingCollisionResult ClassifyCollision(Point3d center, double
radius, Entity structure)
        {
            LashingCollisionResult result = new LashingCollisionResult();
            Point3dCollection pts = new Point3dCollection();

            // Sử dụng cung tròn ảo dạng Arc2d thay vì vẽ Circle thật lên màn hình
bản vẽ

            Circle virtualCircle = new Circle(center, Vector3d.ZAxis, radius);
            virtualCircle.IntersectWith(structure, Intersect.OnBothOperands, pts,
IntPtr.Zero, IntPtr.Zero);

            if (pts.Count >= 2)
            {
                result.CollisionOccurred = true;
                result.DeltaX = Math.Abs(pts[0].X - pts[1].X);
                result.DeltaY = Math.Abs(pts[0].Y - pts[1].Y);

                if (result.DeltaY > (result.DeltaX * 5.0) + 0.000001)
                    result.CollisionType = LashingCollisionType.Vertical;
                else if (result.DeltaX > (result.DeltaY * 5.0) + 0.000001)
                    result.CollisionType = LashingCollisionType.Horizontal;
                else
                    result.CollisionType = LashingCollisionType.Complex;
            }
            return result;
        }

        // Logic quét cấu kiện Panel liền kề: Khởi tạo vùng cấm ảo hoàn toàn trong
bộ nhớ RAM
        public List<Line3d> CreateVirtualKeepOutZones(SelectionSet adjStructures,
double boundMin, double boundMax, bool isVertical)
        {
            var zones = new List<Line3d>();
            foreach (SelectedObject id in adjStructures)
            {
                using (var ent = id.ObjectId.Open(OpenMode.ForRead) as Polyline)
                {
                    if (ent == null) continue;
                    Extents3d box = ent.GeometricExtents;

```

```

        double dx = Math.Abs(box.MaxPoint.X - box.MinPoint.X);
        double dy = Math.Abs(box.MaxPoint.Y - box.MinPoint.Y);

        // Khớp logic điều kiện tỉ lệ ADJACENT_STRUCTURE_FILTER_RATIO =
300

        if (isVertical && dy > 300.0 * dx)
        {
            zones.Add(new Line3d(new Point3d(box.MinPoint.X, boundMin,
0), new Point3d(box.MinPoint.X, boundMax, 0)));
            zones.Add(new Line3d(new Point3d(box.MaxPoint.X, boundMin,
0), new Point3d(box.MaxPoint.X, boundMax, 0)));
        }
        else if (!isVertical && dx > 300.0 * dy)
        {
            zones.Add(new Line3d(new Point3d(boundMin, box.MinPoint.Y,
0), new Point3d(boundMax, box.MinPoint.Y, 0)));
            zones.Add(new Line3d(new Point3d(boundMin, box.MaxPoint.Y,
0), new Point3d(boundMax, box.MaxPoint.Y, 0)));
        }
    }
    return zones;
}
}
}

```

B. `GridGenerationService.cs` - Tự Động Rải Lưới & Xử Lý Khu Vực Đặc Biệt (Phase 3)

```
namespace SDS.Services
{
    public class GridGenerationService
    {
        // Kế thừa hoàn toàn hàm PerformSpecialAreaAdjustment_Phase3 sang cấu trúc .NET
        public void RegenerateSpecialArea(Circle startHole, Circle endHole, Polyline boundary, double spacing, double offset, bool isVertical)
        {
            Point3d startCenter = startHole.Center;
            Point3d endCenter = endHole.Center;

            // 1. Tự động xác định hướng phát triển đối xứng (Auto Direction Logic)
            Point3d midPtSelection = new Point3d((startCenter.X + endCenter.X) / 2, (startCenter.Y + endCenter.Y) / 2, 0);
            Extents3d panelBox = boundary.GeometricExtents;
            Point3d panelCenter = new Point3d((panelBox.MinPoint.X + panelBox.MaxPoint.X) / 2, (panelBox.MinPoint.Y + panelBox.MaxPoint.Y) / 2, 0);

            int genDir = 1;
            if (isVertical)
                genDir = (midPtSelection.Y - panelCenter.Y) >= 0 ? 1 : -1;
            else
                genDir = (midPtSelection.X - panelCenter.X) >= 0 ? 1 : -1;

            // 2. Thực hiện tính toán giao điểm đường biên ảo và xóa / vẽ cuộn chiếu an toàn...
            // 3. Áp dụng bộ lọc kích thước tiêu chuẩn (Bỏ qua không tạo Dim nếu dVal == spacingY / spacingX tiêu chuẩn)
        }
    }
}
```

4. KHUNG GIAO DIỆN WPF TÍCH HỢP PALETTE (📁 UI/LASHINGHOLEPALETTE.XAML)

Thiết kế front-end chuẩn giao diện tối Dark Theme của AutoCAD, gán thuộc tính điều khiển mượn mà:

```

<UserControl x:Class="SDS.UI.LashingHolePalette"
    xmlns="http://schemas.microsoft.com/winfx/2000/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2000/xaml"
    Background="#1E1E1E" Width="350">
    <StackPanel Margin="15">
        <TextBlock Text="LASHING HOLE GENERATOR" Foreground="#90CDF4"
            FontWeight="Bold" Margin="0,0,0,15"/>
        <Label Content="Panel Name:" Foreground="White"/>
        <TextBox x:Name="txtPanelName" Text="PNL_01" Background="#2D2D2D"
            Foreground="White" Margin="0,0,0,10"/>
        <Label Content="Hole Diameter (D):" Foreground="White"/>
        <TextBox x:Name="txtHoleDiameter" Text="55" Background="#2D2D2D"
            Foreground="White" Margin="0,0,0,10"/>
        <CheckBox x:Name="chkAutomatic" Content="Automatic Generate"
            Foreground="White" Margin="0,5,0,5"/>
        <CheckBox x:Name="chkCheckAdjacent" Content="Check Adjacent Panel"
            Foreground="White" Margin="0,5,0,15"/>
        <Button x:Name="cmdOK" Content="Create Lashing Hole" Background="#2B6CB0"
            Foreground="White" Height="30"/>
    </StackPanel>
</UserControl>

```

5. ĐĂNG KÝ LỆNH CAD BẢN XỨ HỆ THỐNG (` COMMANDS/LASHINGHOLECOMMANDS.CS`)

Quản lý luồng giao dịch Transactions và thiết lập giá trị hệ thống `DIMSCALE = 25` tự động, xử lý tìm kiếm diện rộng không phụ thuộc vào bộ nhớ đệm màn hình:


```

using Autodesk.AutoCAD.ApplicationServices;
using Autodesk.AutoCAD.DatabaseServices;
using Autodesk.AutoCAD.EditorInput;
using Autodesk.AutoCAD.Runtime;

namespace SDS.Commands
{
    public class LashingHoleCommands
    {
        [CommandMethod("CFS_CreateLashingHole")]
        public void RunCreateLashingHole()
        {
            Document doc = Application.DocumentManager.MdiActiveDocument;
            Database db = doc.Database;
            Editor ed = doc.Editor;

            using (Transaction tr = db.TransactionManager.StartTransaction())
            {
                // Khóa cứng biến tỷ lệ DIMSCALE = 25 như macro nguyên bản
                Application.SetSystemVariable("DIMSCALE", 25.0);

                // Gọi thực thi luồng UI và xử lý lỗi tuần tự tại đây...
                tr.Commit();
            }
        }
    }
}

```

LƯU Ý QUAN TRỌNG KHI TRIỂN KHAI TRÊN .NET API:

1. **Quét cấu kiện diện rộng:** Phương thức `Editor.SelectCrossingWindow` trong .NET quét trực tiếp vào database hình học nên không cần chạy hàm Zoom màn hình (`ZoomWindow`) như VBA.
2. **An toàn bộ nhớ RAM:** Các đối tượng ảo phục vụ kiểm tra va chạm (như `virtualCircle` , `Line3d`) không được đẩy vào Database, do đó không tạo ra các thực thể rác màu hồng trên bản vẽ của người dùng nếu xảy ra sự cố sập luồng tính toán.